

CS EDUCATION

🔍

캐시(Cache)

26.6.26

박현정



교육 목표

1. 캐시의 정의, 필요성, 동작 원리를 이해한다.
2. 캐시 히트, 캐시 미스에 대해 이해한다.
3. 캐시 교체 정책, 무효화 정책을 이해한다.
4. 캐시가 사용되는 사례를 이해한다.

캐시(Cache)란 무엇일까?



매일 아침 딸기를 3개 먹는 사람이 있다.

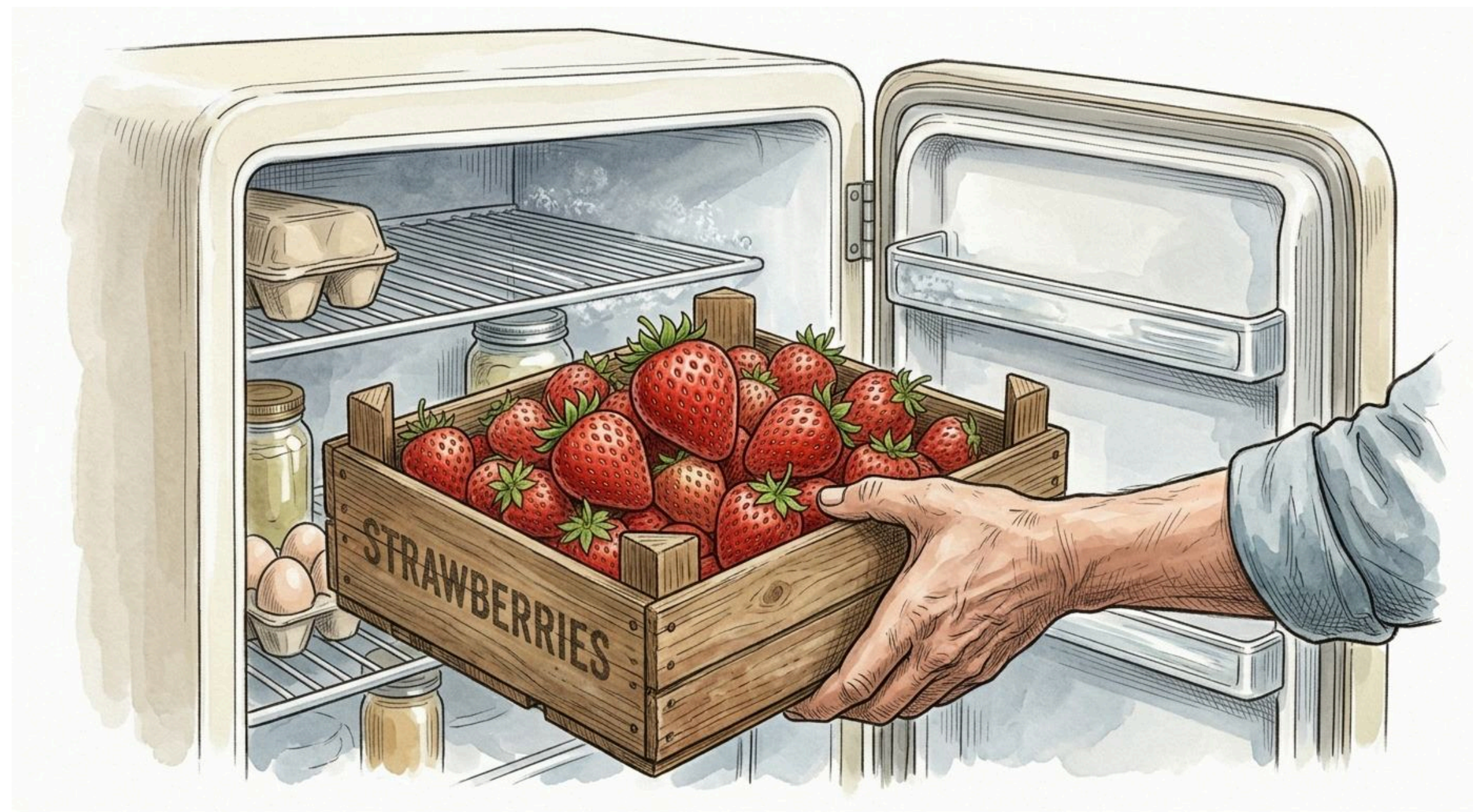
캐시(Cache)란 무엇일까?



X 3

매일 아침 딸기 밭에 가서 딸기 1개를 따와 먹는 일을 3번 반복한다면?
⇒ 매우 비효율적이다.

캐시(Cache)란 무엇일까?



딸기 한 박스를 사서 냉장고에 넣어두고, 매일 아침 3개씩 꺼내먹는다.

캐시(Cache)란 무엇일까?

자주 사용하는 물건은
가까운 곳에 두는 것이 훨씬 효율적이다.



딸기 한 박스를 사서 냉장고에 넣어두고, 매일 아침 3개씩 꺼내먹는다.

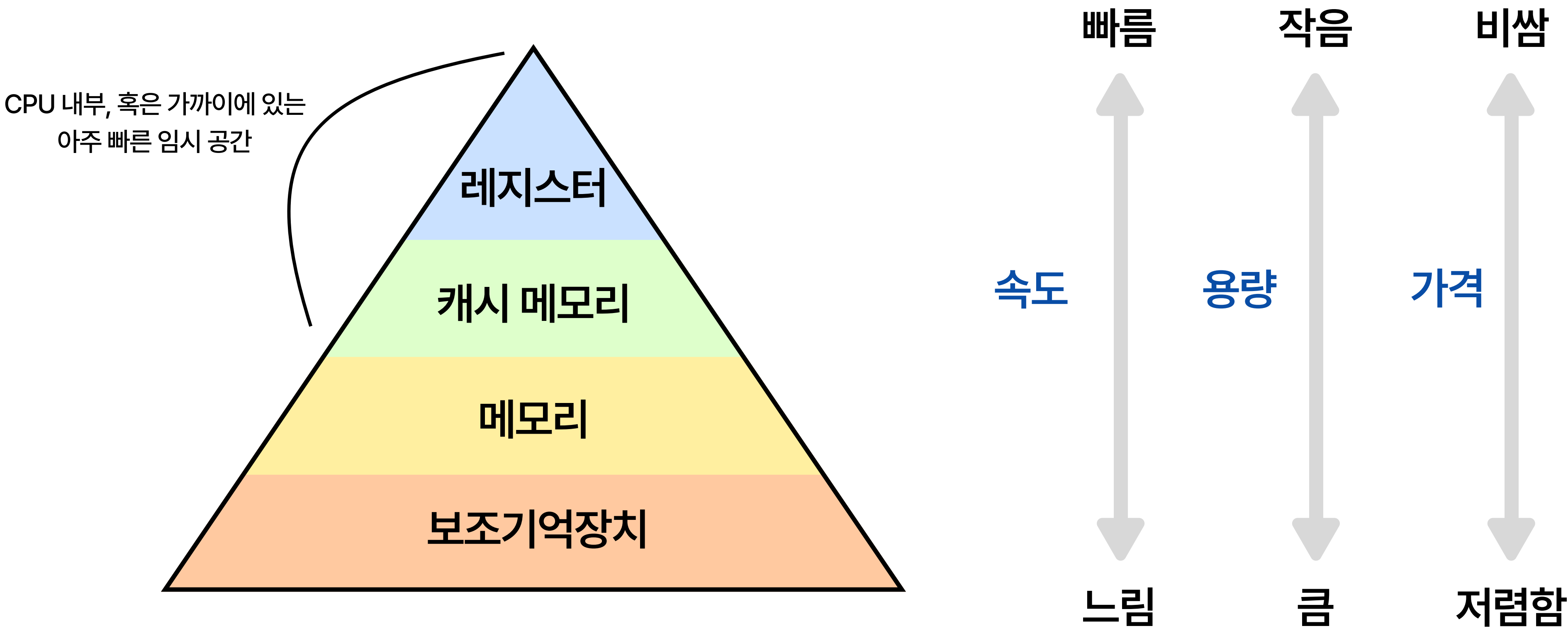




컴퓨터의 데이터 접근 방식도 비슷하다.

자주 사용하는 데이터를 가까운 곳에 두어
빠르게 불러오는 구조를 가지고 있다.

컴퓨터 내 저장장치의 계층 구조



캐시(Cache)란 무엇일까?



자주 사용하는 데이터나 값을
미리 복사해 놓는 임시 저장소

≡ 딸기 예시에서의 냉장고

소프트웨어 또는 하드웨어의 효율성을 높이기 위해 사용되는
여러 유형의 임시 메모리 저장소를 지칭한다.

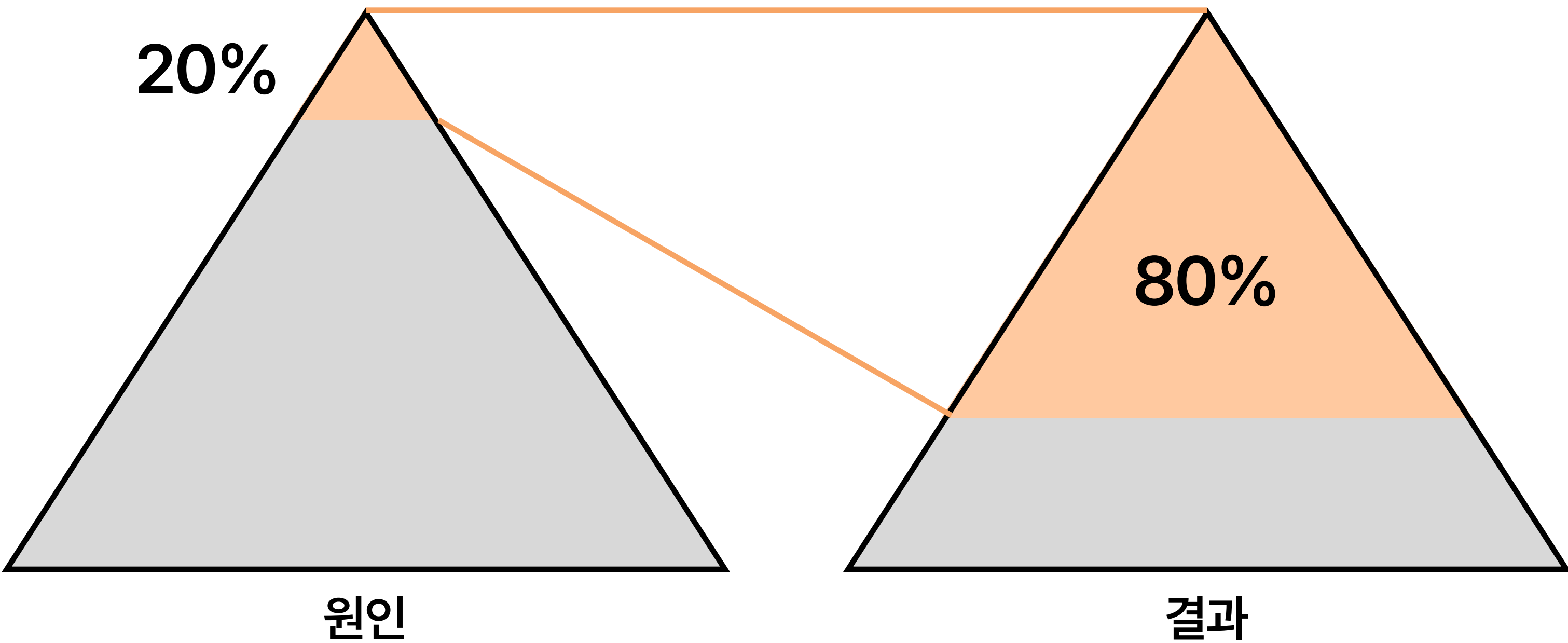
캐시(Cache)란 무엇일까?

캐시로 처리한다

=

결과를 저장해두고, 다시 똑같은 요청이 오면 그 요청에 대해서
DB 또는 API를 참조하지 않고 캐시를 참조하여 요청을 처리한다

파레토의 법칙



80%의 결과는 20%의 원인으로 인해 발생한다

ex. 어떤 가게의 80% 매출이 20%의 단골들에게서 나온다

이 핵심 20% 데이터는 어떻게 선별하는가?
(= CPU가 사용할 법한 데이터는 어떻게 예측하는 걸까?)

➡ **지역성의 원리**에 따라

지역성의 원리

지역성이란?

기억장치 내의 정보를 균일하게 Access하는 것이 아닌
어느 한 순간에 특정 부분을 집중적으로 참조하는 특성

지역성의 원리

시간 지역성

최근에 접근했던 메모리 공간에
다시 접근하려는 경향

for문의 조건 변수 i
⇒ 반복문을 돌 때마다 변수 i를 계속 읽고
쓴다

공간 지역성

접근한 메모리 공간의 근처에
접근하려는 경향

array[0] 접근 후
array[1], array[2]...



지역성의 원리

시간 지역성

공간 지역성

이 두 가지 성질 덕분에
'자주 쓰는 데이터를 미리 빠른 곳에 가져다 놓는다'는
캐시 전략이 실제로 효과를 발휘할 수 있다

캐시의 동작 원리

원본 저장소

DB / 서버 조회

→ 캐시에 저장

다음엔 히트 가능

→ 반환

느림 (<수십ms)

→ 응답 완료

캐시 미스(Cache Miss)

데이터 요청

캐시 확인

캐시 히트(Cache Hit)

바로 반환

빠름 (<1ms)

→ 응답 완료

캐시 히트 & 캐시 미스

캐시 히트 (Cache Hit)

요청한 데이터가 캐시에 존재하여
원본 저장소에 접근하지 않고 바로 반환되는 경우

캐시 미스 (Cache Miss)

요청한 데이터가 캐시에 존재하지 않아
원본 저장소에서 직접 데이터를 가져와야 하는 경우

캐시 적중률

캐시가 히트되는 비율(Cache Hit Ratio)

캐시 히트 횟수

캐시 히트 횟수 + 캐시 미스 횟수

예를 들어 100번의 요청 중 90번 캐시에서 응답했으면
Cache Hit Ratio = 90%

⇒ 높을 수록 캐시 성능이 좋다.

교체 정책

용량이 적은 캐시가 꼭 찾을 때 어떤 데이터를 버려야 할까?

알고리즘

FIFO (First In First Out)

LRU (Least Recently Used)

LFU (Least Frequently Used)

기준

입력 순서

최근 사용 시간

사용 빈도

핵심

먼저 들어온 데이터를 먼저 제거

가장 오랫동안 안 쓴 데이터를 제거

가장 적게 사용된 데이터를 제거

캐시 무효화 정책

캐시에 저장된 기존의 오래된 데이터를 무효화(삭제하거나 갱신)하여
사용자가 항상 최신 데이터를 볼 수 있도록 보장하는 프로세스

시간 기반 무효화 = TTL (Time To Live)

변경 기반 무효화

수동 무효화

만료 시간을 설정해두고, 시간이 지나면 자동으로 캐시를 삭제

원본 데이터가 변경되는 순간을 감지해서
자동으로 업데이트하거나 삭제

필요에 따라 애플리케이션 로직을 통해
데이터를 수동으로 무효화

캐시가 사용되는 사례(1)

CPU 캐시 메모리

초고속 프로세서(CPU)와 상대적으로 속도가 느린
메인 메모리(RAM) 사이의 속도 차이를 줄여
병목 현상을 해결하는 임시 고속 메모리

CPU는 프로그램을 실행하기 위하여 메모리에 접근하여 코드
나 데이터를 가져온다.

메모리에서 데이터를 받아오는 속도가 느리다면 병목 발생

- CPU 내부에 SRAM 기반의 초고속 저장 장치를 탑재
- 자주 쓰는 데이터를 CPU 가까이에 미리 올려둠

L1, L2, L3 캐시



캐시가 사용되는 사례(2)

데이터베이스 캐시

DB는 데이터를 디스크에서 읽어오기 때문에 응답이 느리다
요청이 몰리면 DB 부하 ↑

자주 요청되는 쿼리 결과를 메모리에 올려두는 캐싱 전략 사용

Redis 같은 캐시 서버를 DB 앞단에 두고, 요청이 오면 DB까지
가기 전에 Redis에서 먼저 응답하도록 구성



캐시가 사용되는 사례(3)

CDN (Content Delivery Network)



웹 콘텐츠를 세계 곳곳에 있는 여러 서버에 분산하여 저장하는
분산 서버 네트워크 시스템

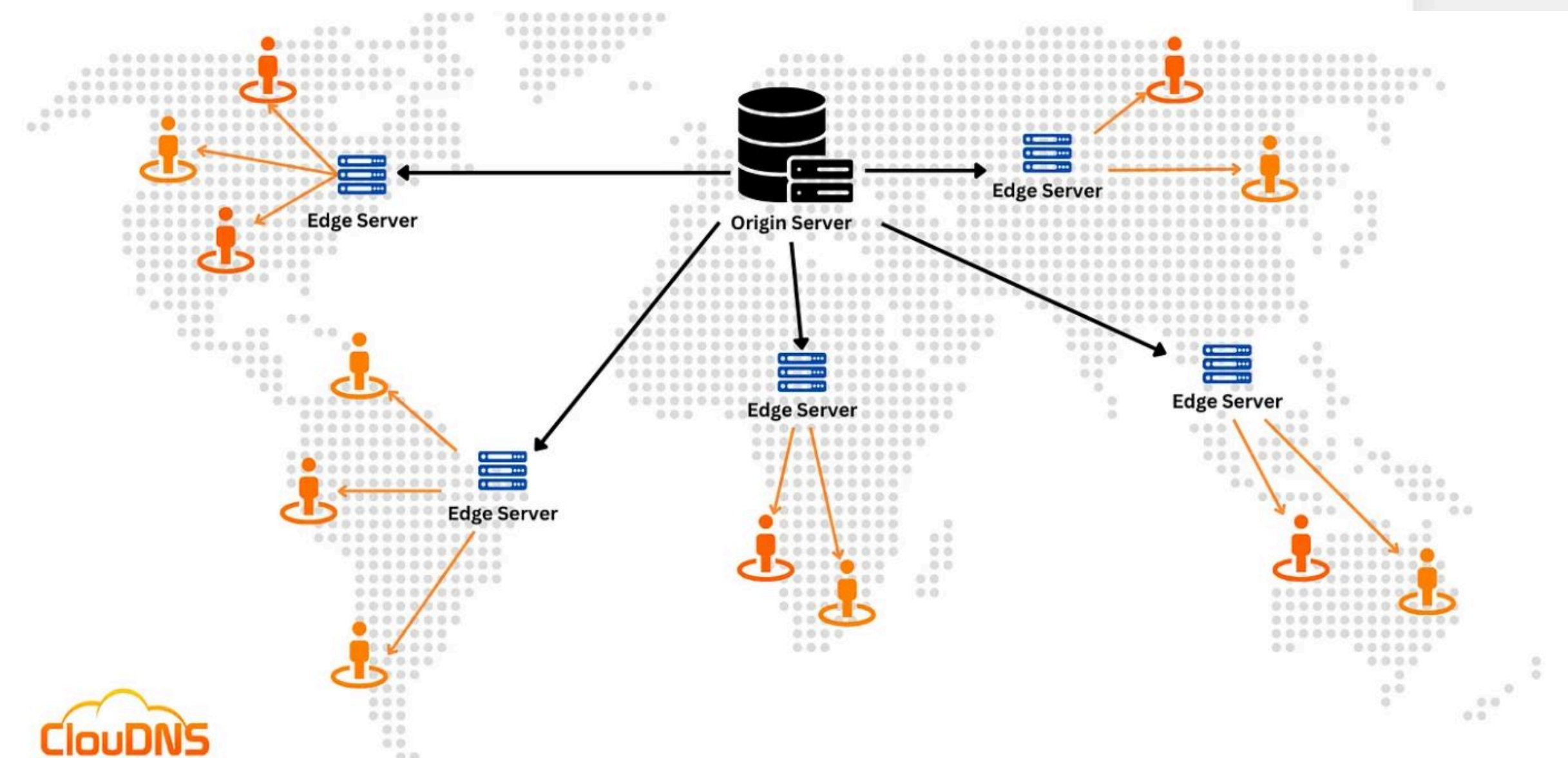
유튜브 서버는 미국에 있다.

한국에서 매번 미국 서버까지 접속한다면?

→ 전 세계 각지에 캐시 서버를 분산 배치

→ 사용자와 가까운 서버에서 콘텐츠를 제공

Cloudflare, AWS CloudFront, Google Cloud CDN 등



캐시가 사용되는 사례(4)

웹 캐시 (브라우저 캐시)

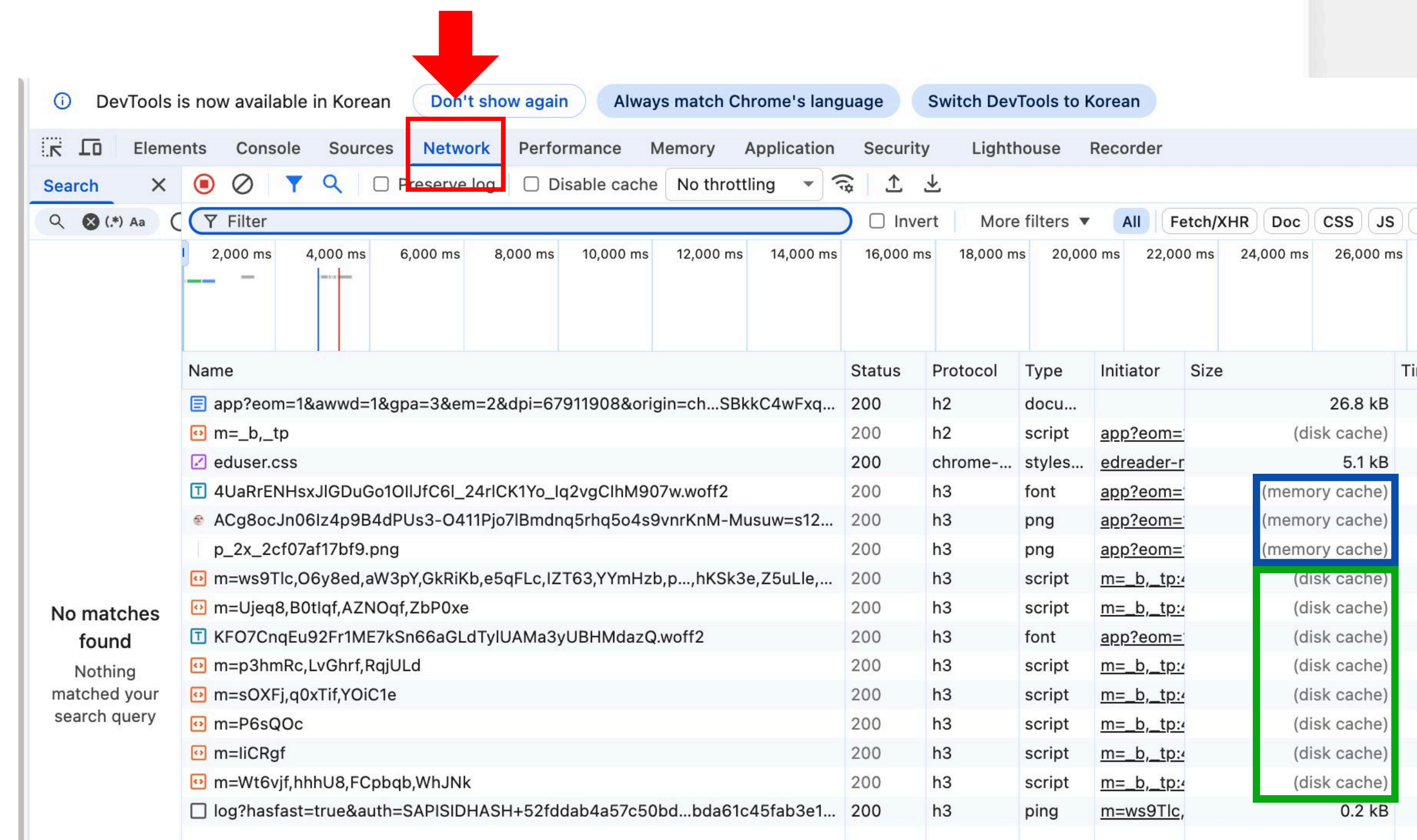
웹사이트 접속 시 HTML, CSS, JS, 이미지를 로컬 디스크에 저장해두고 재사용

memory cache

: RAM에 저장, 브라우저 종료 시 사라짐

disk cache

: SSD에 저장, 브라우저 종료 후에도 유지



정리

- 캐시는 자주 사용하는 데이터를 빠른 곳에 미리 복사해두는 임시 저장소이다.
- 캐시 히트는 캐시에 데이터가 있어서 바로 반환되는 경우, 캐시 미스는 없어서 원본 저장소까지 가야 하는 경우를 지칭한다.
- 캐시가 가득 찼을 때는 교체 정책(FIFO, LRU, LFU)으로 어떤 걸 버릴지 결정하고, 오래된 데이터는 무효화 정책(TTL, 변경 기반, 수동)으로 최신 상태를 유지한다.
- 캐시는 CPU 메모리, DB 캐시, CDN, 브라우저 캐시 등 우리 일상 곳곳에서 사용되고 있다.